

Task 1: Check the current SELinux status on your Linux system using the getenforce and sestatus commands, and write down what each output means.

Ans :-

Command 1

getenforce

Example Output

Enforcing

Meaning

- **Enforcing** → SELinux is active and enforcing all security policies.
- **Permissive** → SELinux does not block actions but logs violations.
- **Disabled** → SELinux is completely turned off.

Command 2

sestatus

Example Output

SELinux status: enabled
SELinuxfs mount: /sys/fs/selinux
SELinux root directory: /etc/selinux
Loaded policy name: targeted
Current mode: enforcing
Mode from config file: enforcing
Policy MLS status: enabled
Policy deny_unknown status: allowed
Memory protection checking: actual (secure)
Max kernel policy version: 33

Meaning

Field	Meaning
SELinux status	Whether SELinux is enabled
Current mode	Current running mode (Enforcing/Permissive)

Field	Meaning
Mode from config file	Mode after next reboot
Loaded policy name	Security policy being used (usually targeted)
SELinux root directory	Location of SELinux configuration files

Task 2: Switch SELinux to permissive mode temporarily using the setenforce command, then try creating a new file in /var/www/html and note any differences in error messages or permissions compared to enforcing mode.
Hint: Use setenforce 0 for permissive and setenforce 1 to return to enforcing.

Ans :-

Check current mode

getenforce

Output

Enforcing

Switch to permissive mode

sudo setenforce 0

Verify

getenforce

Output

Permissive

Create a file

sudo touch /var/www/html/test.html

or

echo "Hello World" | sudo tee /var/www/html/test.html

Difference

Enforcing Mode

- SELinux blocks unauthorized access.
- You may receive **Permission denied** if the SELinux policy forbids the operation.
- Actions are denied and logged.

Permissive Mode

- Operation succeeds.
- SELinux only logs violations.
- No blocking occurs.

Return to Enforcing Mode

`sudo setenforce 1`

Verify

`getenforce`

Output

Enforcing

Task 3: List all files in `/var/www/html` and use the `ls -Z` command to display their SELinux security contexts. Change the context of one file to `httpd_sys_content_t` using `chcon` and verify the change.
Hint: Use `chcon --type=httpd_sys_content_t filename`

Ans :-

List files with SELinux context

`ls -Z /var/www/html`

Example

`-rw-r--r--. root root system_u:object_r:httpd_sys_content_t:s0 index.html`

`-rw-r--r--. root root unconfined_u:object_r:default_t:s0 test.html`

Explanation

Part	Meaning
<code>system_u</code>	SELinux user

Part	Meaning
object_r	Role
httpd_sys_content_t	Type (most important)
s0	Security level

Change the context

```
sudo chcon --type=httpd_sys_content_t /var/www/html/test.html
```

Verify

```
ls -Z /var/www/html/test.html
```

Example

```
-rw-r--r--. root root unconfined_u:object_r:httpd_sys_content_t:s0 test.html
```

The file is now labeled so the Apache web server can read it.

Task 4: A user is unable to upload images to a folder for their food review blog (like Zomato) hosted in /srv/foodie/images. Use the audit2why and audit2allow tools to analyze SELinux denial logs and suggest a solution to allow uploads.
Hint: Check /var/log/audit/audit.log for recent denials.

Ans :-

Suppose uploads are failing in:

/srv/foodie/images

Step 1: View recent SELinux denials

```
sudo ausearch -m AVC -ts recent
```

or

```
sudo cat /var/log/audit/audit.log | grep AVC
```

Example

```
type=AVC msg=audit(...)
```

```
avc: denied { write } for pid=2356
comm="httpd"
name="images"
tcontext=system_u:object_r:default_t:s0
```

Step 2: Explain why

```
sudo audit2why < /var/log/audit/audit.log
```

Example

```
type=AVC
```

Was caused by:

Missing type enforcement (TE) allow rule.

Meaning:

- SELinux blocked Apache (httpd) because the directory has the wrong security context or there is no policy allowing write access.

Step 3: Generate a policy

```
sudo audit2allow -a
```

Example

```
allow httpd_t default_t:dir write;
```

```
allow httpd_t default_t:file create;
```

To build and install a local module:

```
sudo audit2allow -a -M foodie_upload
```

```
sudo semodule -i foodie_upload.pp
```

Step 4 (Recommended Solution)

Instead of creating a custom policy, assign the proper SELinux context for upload directories:

```
sudo semanage fcontext -a -t httpd_sys_rw_content_t "/srv/foodie/images(/.*)"?"
```

Apply it:

```
sudo restorecon -Rv /srv/foodie/images
```

Verify:

```
ls -Zd /srv/foodie/images
```

Expected output:

```
system_u:object_r:httpd_sys_rw_content_t:s0 /srv/foodie/images
```

This context allows the Apache web server to read and write files in the directory